SEMANA 03

Listas Enlazadas Dobles y Circulares

< Datos />
{ listas }
→ nodos
← prev

Estructura de Datos con C#

Curso: Estructura de Datos (SIST1201A) | UPN 2026-1

Docente: Dr. Eric Gustavo Coronel Castillo

PRESENTACIÓN DE LA SESIÓN

🎯 **Logro:** Al término de la sesión, el estudiante implementa algoritmos de listas doblemente enlazadas y circulares en C#, aplicando buenas prácticas de programación.

Temas de la sesión

1. Listas doblemente enlazadas: definición y estructura del nodo
2. Lista circular simplemente enlazada
3. Lista circular doblemente enlazada
4. Operaciones CRUD en listas dobles
5. Implementación en C# con buenas prácticas

Competencias que desarrolla

- 💡 Pensamiento creativo y crítico
- 🔧 Resolución de problemas algorítmicos
- 📐 Conocimientos de Ingeniería de Software
- 🔍 Análisis de estructuras de datos dinámicas

REFLEXIONA ANTES DE COMENZAR

1

¿Cuál es la diferencia entre una lista simple y una doble?

Piensa en la cantidad de punteros por nodo.

2

¿Qué ventaja tiene poder recorrer la lista hacia atrás?

Considera operaciones de eliminación y búsqueda inversa.

3

¿Qué problema puede ocurrir en una lista circular?

¿Cómo sabe el algoritmo cuándo detenerse?

Responde estas preguntas en tu cuaderno antes de continuar.

LISTAS DOBLEMENTE ENLAZADAS

Definición: Cada nodo tiene tres campos: **Anterior (prev)**, **Dato** y **Siguiente (next)**. Permite recorrer la lista en ambas direcciones.

Estructura del Nodo:

prev: apunta al nodo anterior (null si es el primero) **next**: apunta al nodo siguiente (null si es el último)

Lista con tres nodos enlazados:

Cormen et al. (2022). Introduction to Algorithms (4th ed., p. 258). MIT Press. https://mitpress.mit.edu/9780262046305/

COMPLEJIDAD TEMPORAL — NOTACIÓN Big-O

¿Cuánto cuesta ejecutar cada operación en una lista doblemente enlazada?

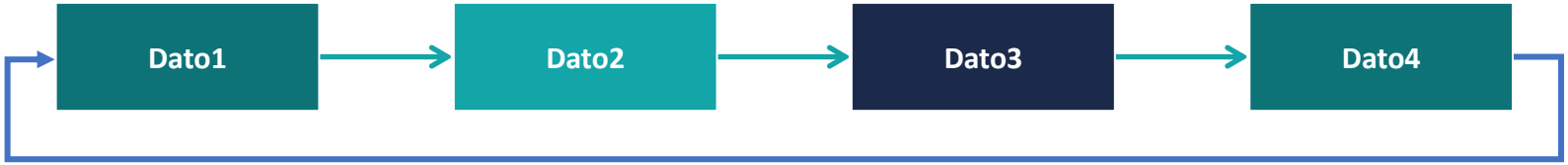
Operación	Lista Doble	Condición
Insertar al inicio	O(1)	Modificar head y prev del primer nodo
Insertar al final	O(1)	Si se mantiene referencia al tail
Insertar en medio	O(n)	Hay que recorrer para encontrar posición
Eliminar (nodo conocido)	O(1)	Solo se ajustan los punteros next/prev
Eliminar por valor	O(n)	Primero hay que buscar el nodo
Búsqueda	O(n)	Recorrido secuencial (no hay índice)
Acceso por índice	O(n)	No hay acceso directo como en arrays

⚠️ Ventaja clave: eliminar un nodo conocido es O(1) porque cada nodo sabe quién es su predecesor (prev), a diferencia de la lista simple.

Cormen et al. (2022). Introduction to Algorithms (4th ed., cap. 10). MIT Press. https://mitpress.mit.edu/9780262046305/

LISTA CIRCULAR SIMPLEMENTE ENLAZADA

Definición: Es una lista enlazada simple donde el puntero **next** del último nodo (cola) apunta de regreso al primer nodo (cabeza), formando un "anillo" continuo.



← ← ← ← ← ← ← *flecha circular (el último apunta al primero)* ← ← ← ←

⚠️ **RIESGO CRÍTICO — Ciclo infinito:** Al recorrer una lista circular, el algoritmo nunca encuentra un puntero null. Sin una condición de parada explícita (verificar si el puntero regresó al nodo inicial), el bucle se ejecutará eternamente.

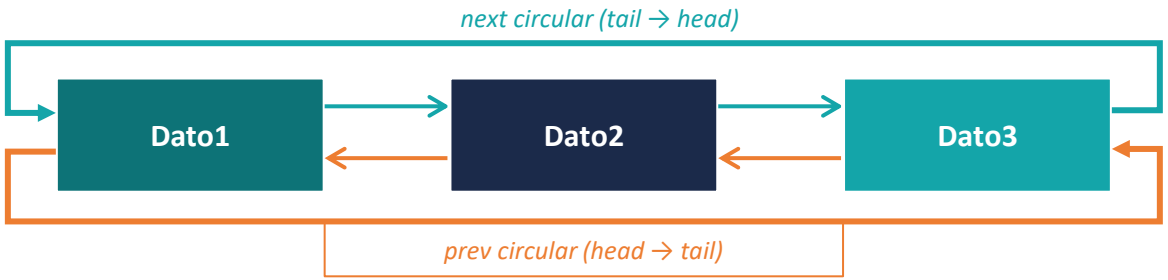
Casos de uso reales:

- Gestión de turnos en procesos concurrentes (Round Robin)
- Reproducción en bucle de listas de música o video
- Buffers circulares en sistemas de comunicación

Sedgewick, R. & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley. https://algs4.cs.princeton.edu/home/

LISTA CIRCULAR DOBLEMENTE ENLAZADA

Definición: Combina las características de la lista doble y la lista circular. El **next** del último nodo apunta al primero, y el **prev** del primero apunta al último. Permite recorrido circular en ambas direcciones.



Comparación de tipos de listas:

Tipo	Dirección	Puntero final
Simple	Un sentido	→ null
Doble	Dos sentidos	prev=null / next=null
Circular Simple	Un sentido (anillo)	next → head
Circular Doble	Dos sentidos (anillo)	next→head / prev→tail

Microsoft. (2024). LinkedList<T> Class. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.linkedlist-1

OPERACIONES EN LISTAS DOBLEMENTE ENLAZADAS

➕ Insertar

- Al inicio: ajusta head.prev → nuevo nodo
- Al final: ajusta tail.next → nuevo nodo
- En medio: modifica prev y next del nodo adyacente
- Costo: O(1) si se conoce la posición

🔄 Recorrer / Mostrar

- Hacia adelante: start → next → ... → null
- Hacia atrás: tail → prev → ... → null
- En circular: detener cuando ptr == head
- Costo: O(n)

🔍 Buscar

- Recorrido secuencial comparando el dato
- Puede iniciarse desde head o tail
- No hay índice directo (diferente al array)
- Costo: O(n) en el peor caso

🗑️ Eliminar

- Conectar prev.next → nodo.next
- Conectar next.prev → nodo.prev
- Si nodo es head o tail: actualizar referencias
- Costo: O(1) si se tiene referencia al nodo

Hernández Bejarano, M. & Baquero Rey, L. E. (2022). Estructuras de datos: fundamentación práctica. https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/230581

IMPLEMENTACIÓN EN C# — Clase NodoDoble<T>

```
// Principio SRP: el nodo solo almacena datos
public class NodoDoble<T>
{
    public T Dato { get; set; }
    public NodoDoble<T>? Siguiente { get; set; }
    public NodoDoble<T>? Anterior  { get; set; }

    public NodoDoble(T dato)
    {
        Dato      = dato;
        Siguiente = null;
        Anterior  = null;
    }
}
```

Puntos clave del diseño

🔷 **Genérico <T>** La clase acepta cualquier tipo de dato: int, string, objetos.

🔷 **Nullable (?)** Siguiente y Anterior pueden ser null. Cumple con el principio de seguridad de tipos.

🔷 **SRP** El nodo solo gestiona sus datos y punteros, no la lógica de la lista.

🔷 **Properties** Se usan propiedades C# (get/set) en lugar de campos públicos. Buena práctica.

Microsoft. (2024). Clases y objetos en C#. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/csharp/fundamentals/types/classes

IMPLEMENTACIÓN EN C# — Clase ListaDoble<T> (Insertar y Recorrer)

```
public class ListaDoble<T>
{
    private NodoDoble<T>? _cabeza;
    private NodoDoble<T>? _cola;

    // Insertar al final — O(1)
    public void InsertarAlFinal(T dato)
    {
        var nuevo = new NodoDoble<T>(dato);
        if (_cabeza is null) { _cabeza = _cola = nuevo; return; }
        nuevo.Anterior  = _cola;
        _cola!.Siguiente = nuevo;
        _cola           = nuevo;
    }

    // Recorrer hacia adelante — O(n)
    public void MostrarAdelante()
    {
        var actual = _cabeza;
        while (actual is not null)
        {
            Console.Write($"{actual.Dato} → ");
            actual = actual.Siguiente;
        }
    }
}
```

Ceballos Sierra, F. (2014). Microsoft C#: Curso de Programación (2.ª ed.). RA-MA. https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/106417

IMPLEMENTACIÓN EN C# — Eliminar y Lista Circular

Eliminar un nodo:

```
public bool Eliminar(T dato)
{
  var actual = _cabeza;
  while (actual is not null)
  {
    if (actual.Dato!.Equals(dato))
    {
      // Ajustar puntero anterior
      if (actual.Anterior != null)
        actual.Anterior.Siguiente
          = actual.Siguiente;
      else _cabeza = actual.Siguiente;
      // Ajustar puntero siguiente
      if (actual.Siguiente != null)
        actual.Siguiente.Anterior
          = actual.Anterior;
      else _cola = actual.Anterior;
      return true; // eliminado
    }
    actual = actual.Siguiente;
  }
  return false; // no encontrado
}
```

Recorrer lista circular (condición de parada):

```
// ⚠️ Siempre verificar ptr == head
public void RecorrerCircular()
{
    if (_cabeza is null) return;
    var actual = _cabeza;
    do
    {
        Console.Write($"{actual.Dato} ");
        actual = actual.Siguiente!;
    } while (actual != _cabeza); // ← parada
}
```

⚠️ Sin el while (actual != _cabeza), el bucle nunca termina → CICLO INFINITO. Es el error más común al trabajar con listas circulares.

💡 **.NET ya incluye: `LinkedList<T>`** — Implementación oficial de lista doblemente enlazada. En producción se recomienda usar la clase del framework.

Microsoft. (2024). LinkedList<T> Class. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.linkedlist-1

RETOS — EJERCICIOS PROPUESTOS

01 Básico

Agenda de contactos

Implementa una ListaDoble<Contacto> donde Contacto tiene Nombre y Teléfono. Agrega los métodos Insertar, MostrarAdelante y MostrarAtras.

02 Intermedio

Gestor de historial

Simula el historial de un navegador web. Al hacer clic en Atrás se usa el puntero Anterior, y al Adelante el puntero Siguiente. Implementa en C# con menú de consola.

03 Avanzado

Cola circular de turnos

Modela el algoritmo Round Robin usando una lista circular. Cada nodo representa un proceso con Nombre y TiempoRestante. Itera la lista en anillo descontando tiempo por ciclo.

04 Extra

Buscar y eliminar duplicados

Dada una ListaDoble<int> con valores repetidos, implementa el método EliminarDuplicados() que deje solo la primera ocurrencia de cada valor. Analiza la complejidad de tu solución.

REFERENCIAS BIBLIOGRÁFICAS

01 Ceballos Sierra, F. (2014). Microsoft C#: Curso de Programación (2.ª ed.). RA-MA. https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/106417

02 Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

03 Hernández Bejarano, M., & Baquero Rey, L. E. (2022). Estructuras de datos: fundamentación práctica. https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/230581

04 Microsoft. (2024). LinkedList<T> Class. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.linkedlist-1

05 Microsoft. (2024). Guía de programación en C#. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/csharp/programming-guide/

06 Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional. https://algs4.cs.princeton.edu/home/

07 Uceda Martos, P. J. (2026). Sílabo del curso Estructura de Datos (SIST1201A). Universidad Privada del Norte, Período 2026-1. https://www.upn.edu.pe

RESUMEN DE LA SESIÓN

¿Qué aprendimos hoy?

① La lista doblemente enlazada usa punteros prev y next, permitiendo recorrido bidireccional.

② Insertar y eliminar un nodo conocido cuesta O(1); buscar cuesta O(n).

③ La lista circular simple conecta la cola con la cabeza; siempre necesita condición de parada.

④ La lista circular doble combina ambas capacidades y es la base de LinkedList<T> en .NET.

⑤ En C# implementamos NodoDoble<T> y ListaDoble<T> siguiendo principios SOLID.

Próxima sesión: Semana 04 — Análisis de problemas complejos con herramientas de IA